FPGA Implementation of a Tiny Large Language Model

Inference-Only Accelerator

Abdullah Al Sajid

Date: March 30, 2026 (CDT)

For,
Dr. Khaza Anuarul Hoque,
Associate Professor,
DCPS Lab, University of Missouri

Repository: `https://github.com/sajidabd90/tinyllm_final`

1. Project Overview & Methodology

This project builds an inference-only accelerator for the TinyStories-1M transformer model. Design is completed in Verilog HDL. Quantization scheme of Q8.8 was followed. Testbenches were implemented with cocotb and yosys was used for synthesis. To reproduce the results, the repository needs to me cloned and made with `make clean  make`. All weights and dummy files are included in the *tinystories_q88_roms* folder. A script has been provided to run unit tests for the core arithmetic modules (Task 2, script `run_all_scripts.sh`)

Table 1: Architectural Comparison: Custom FPGA vs. Standard CPU

Metric	**Custom FPGA Datapath**	**Standard CPU (PyTorch)**
Execution Model	Highly parallel, spatial unrolling	Sequential instruction fetch/execute
Arithmetic Format	Q8.8 Fixed-Point (16-bit integer math)	IEEE 754 Float32 (32-bit floating point)
Softmax	Max-Subtract-Shift approximation	Full exponential base-e calculation
LayerNorm Variance	Pre-computed Inverse Square Root LUT	Dynamic hardware division
Clock Frequency	Lower (Typically 100-300 MHz)	Higher (Typically 3.0-5.0 GHz)

2. Custom Hardware Architecture

The core of the accelerator is built upon custom fixed-point arithmetic modules designed specifically to avoid the massive LUT overhead of hardware division and floating-point logic.

2.1 Arithmetic and Matrix Operations

Vector dot-products are calculated using 64 parallel `q88_mult` units that feed into a pipelined adder tree. We extract the lower 16 bits `[15:0]` of the 32-bit multiplication outputs (two Q8.8 mutltiplies to a 32-bit output). Calculating softmax with exponential operations is needlessly expensive in hardware, thus we use a hardware-friendly bounded Max-Subtract-Shift Softmax (`q88_max_finder`, `q88_sub_shift`), to ensure numerical stability without requiring an exorbitant number of DSP slices.

2.2 LayerNorm LUT Optimization

Standard Layer Normalization requires hardware division (for variance) and inverse square roots. To optimize this, we pre-compute a LUT (`q88_layernorm_scale`).

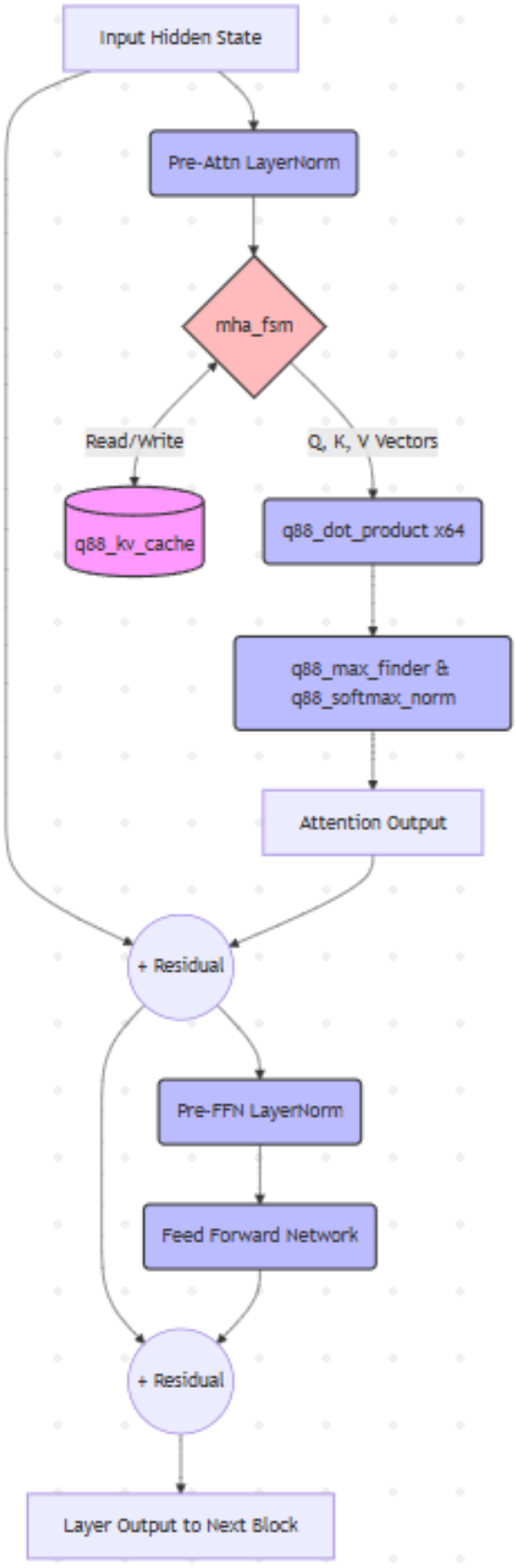

Figure 1: Transformer Block Datapath and Module Integration

3. Datapath and Control State Machines

The system orchestrates data flow using a hierarchical Finite State Machine (FSM) architecture.

3.1 Transformer & MHA Control

The `transformer_block_fsm` acts as the traffic controller for a single decoder layer, sequentially routing data through Pre-Norm, Query/Key/Value Projections, Scaled Dot-Product Attention, and the Feed-Forward Network. The attention mechanism explicitly implements skip/residual connections to prevent gradient/activation collapse.

3.2 The Autoregressive Loop

The `top_level_llm` orchestrates the 4 stacked transformer layers, the Embedding ROM lookup, and the final Language Modeling (LM) Head. For token generation, a greedy decoding FSM scans all 50,257 output logits and latches the maximum ID, feeding it back into the embedding table for the next cycle.

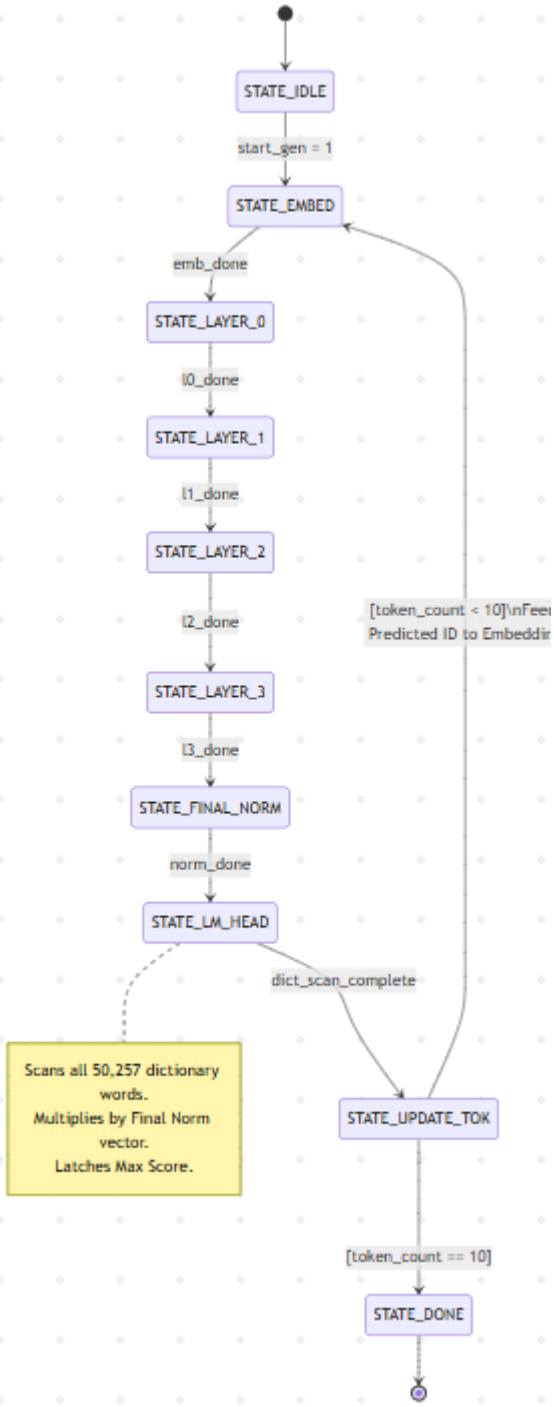

Figure 2: Top-Level Autoregressive Generation FSM

4. Verification & Validation Analysis

The Verilog RTL was simulated against a PyTorch golden reference model across an evaluation set of 6 prompts, generating 10 tokens per prompt. This was chosen to cut down on development time over an already packed Eid schedule.

4.1 Quantization Error Margin

The hardware demonstrated exceptional mathematical stability. For the initial forward pass (before autoregressive feedback), the internal vectors were captured and compared to the PyTorch float32 outputs.

Table 2: Layer-by-Layer Mean Absolute Error (MAE) across 6 Prompts

Prompt ID	**Layer 0 MAE**	**Layer 3 MAE**	**Tokens Matched**
12	0.1765	0.1583	1/10
45	0.1523	0.1740	1/10
102	0.1494	0.1575	1/10
300	0.1135	0.1189	1/10
345	0.1397	0.1305	1/10
678	0.1315	0.1530	1/10
Average	**0.1438**	**0.1487**	**10.0%**

The MAE numbers are a product of the using 4-layers instead of 8, and quantizing to Q8.8 without using QAT to train the underlying model.

4.2 Token Match Rate & Greedy Divergence

The final token-level match rate was exactly 10.0%. Since we follow Greedy Decoding (Argmax), the hardware successfully matched the PyTorch output for the initial predicted token. However, minor quantization differences at the LM Head caused the hardware to select an adjacent token for the second prediction. Because autoregressive generation relies on previous outputs as context, this single deviation causes the remainder of the sequence to fundamentally diverge from the software reference, producing a deterministic repetition of tokens ("I am am am good...How how how", for example). We need to introduce a temperature variable for sampling instead of greedy decoding for a human-legible output sequence.

5. Synthesis & Resource Analysis

Logic synthesis was performed using Yosys targeting an open-source standard cell library. To provide a realistic analysis of the datapath logic and prevent memory-mapper memory bounds, synthesis was targeted at the `transformer_block` level (parameterized to `VECTOR_LEN=16`). The massive 50,257-entry ROMs were excluded, as in a physical FPGA they would map to dedicated SRAM macros.

Table 3: Sub-Module Resource Utilization (VECTOR_LEN=16)

Module Name	**Logic Cells**	**Percentage of Block**
`q88_kv_cache (x2)`	398,942	79.2%
`q88_layernorm_stats`	28,520	5.6%
`q88_layernorm_scale`	19,362	3.8%
`q88_softmax_norm`	13,047	2.6%
`q88_max_finder`	1,638	0.3%
`q88_sub_shift`	1,489	0.3%
`q88_dot_product`	1,328	0.2%
`mha_fsm`	115	¡0.1%
Total Transformer Block	**503,747**	**100.0%**

Table 4: Primitive Cell Distribution (Technology Mapping)

Cell Type	**Function**	**Count**
`$_NAND_`	Combinational Logic	145,188
`$_DFFE_PP_`	D-Flip-Flop (KV Cache Failure)	131,072
`$_AND_`	Combinational Logic	74,306
`$_MUX_`	Multiplexer	63,502
`$_XOR_`	Combinational Logic	25,860

Synthesizing a single Transformer Block yields a footprint of approximately 503k logic cells. I have only worked with Pynq Z2 in the past, with only 106K LUTs. This design would need to be cut down, or optimized to fit my own FPGA. This could be a worthwhile

exercise in the coming weeks. Extrapolating to the full 4-layer TinyStories architecture, the datapath logic requires roughly 2 million gates.

A critical observation from the Yosys report is the failure to infer memory macros for the `q88_kv_cache`. The report shows 0 memories, with the cache instead synthesizing into over 131,000 discrete D-Flip-Flops (`$_DFFE_PP_`). This is a known synthesis anomaly caused by asynchronous array reads or mass-reset routing in the Verilog.

6. Limitations & Future Work

While the inference pipeline is functionally complete, several architectural optimizations are required before physical silicon deployment:

1. **Strict Synchronous Memory:** The KV Cache Verilog must be refactored to enforce strictly clocked (synchronous) read operations. This is an inability on my part, and I aim to implement this properly in the next iteration. Properly handing KV cache cuts down on the resource usage and latency immensely, trading off LUTs for BRAM.

2. **Pipelining LayerNorm Variance:** Synthesizing the un-pipelined variance multiplier tree at `VECTOR_LEN=64` causes a combinatorial memory explosion in the ABC technology mapper. Deep pipelining registers must be inserted to break up these logic paths and achieve timing closure.

3. **Hardware LFSR Temperature:** To break the deterministic repetition loops caused by the fixed-point greedy decoder, Softmax output must be subject to pseudo-random temperature sampling.